Epigenetic Language Models: Lifelong Learning by Intercalating Knowledge into Structural Zeros

Free-Slot Adaptation on a real 8B ternary MoE

Parallax / Chutes.ai

October 11, 2026

AI-assisted draft. Revised through repeated adversarial review; experiments run and checked by the authors. Results are from one real 8B ternary MoE: three headline seeds with different fact values on 40 facts, a 300-fact capacity test, and EP-1 v1, a pre-registered multi-session pilot. LoRA baselines remain single seed. EP-1 v1 failed the concept-gain criterion in every arm; v2 is being designed. The personal-model use case (§8) remains an *untested proposal*. Code: github.com/chutesai/epigenesis.

Abstract

An epigenetic language model keeps its base weights as a frozen genome and writes acquired knowledge into a separate sparse layer occupying exact zeros. Removing that layer restores the base bit-for-bit. FSA targets the pair8 forced-zero mask; serving that design needs a sparse overlay or wider block encoding and remains pending. We test the in-pair variant on a real 8B ternary mixture-of-experts model. On 40 facts, ternary Bop with a local output penalty, a 300k-slot budget and rehearsal recalls 79–80/80 held-out items under teacher forcing and greedy generation across three seeds with different fact values: mean 99.2% on both metrics, with mean general-text ΔNLL −0.0003 (range −0.0015 to +0.0006). It fills 245k–256k slots, adding no parameters and preserving the packed format; position-inclusive storage is 0.32–0.86 MB. All three seeds revoke bit-exactly. LoRA baselines recall 75–78/80 but remain single seed, so their run-to-run spread is unmeasured; 1–2 item gaps are within noise, while 3–4 may matter. Rehearsal and damage evaluation use disjoint sequences from the same corpus. Synthetic in-pair packed patches show no detectable serving slowdown; learned patches are not yet benchmarked. At 300 conflicting facts, FSA reaches only 18.2% recall and fits 22.9% of training facts, versus LoRA's 63.0% TF with fp32 factors and 64.8% at int4. The LoRA factors quantize to int4 without loss on this task. EP-1 v1's five-session self-distillation failed the concept-gain criterion in every arm, including LoRA, but its teacher was later found not to know the probed rules, so v1 was not a valid test. Capacity and multi-session consolidation remain open; EP-1 v2.1 is being designed.

1 Introduction

We use an epigenetic analogy: the base weights are a frozen genome, while acquired knowledge lives in a separate sparse layer that can be removed to recover the base bit-for-bit. Extreme quantization already leaves many weights at exact zero. Can those existing slots carry new knowledge without adding parameters?

Free-Slot Adaptation (FSA). In the pair8 ternary format, at most two of every four aligned pairs are active. The design's write set is the two unselected pairs: 4 of every 8 weights (at least half). This deterministic forced-zero mask ships with the model. Let $\mathcal{S}_0$ be the base's nonzero support and $\mathcal{Z}_0$ its zeros. Freeze the base and shared parameters, then train $\mathcal{R}_d$ only within the forced-zero mask. Store filled positions and signs as a removable patch. Enabling it uses $\mathcal{S}_0 \cup \mathcal{R}_d$; zeroing the mask restores $\mathcal{S}_0$ and the exact pair8 base. Filling unselected pairs can exceed pair8's two-pair limit. Serving needs a sparse overlay beside the packed kernel or blocks re-encoded in a wider format; this path is not built or benchmarked. All existing FSA results instead use the in-pair variant, restricted to zeros inside already-active pairs. This is Mallya and Lazebnik [2018]'s mechanism, with the reserve supplied by the compression format rather than a pruning pass.

What we claim. On 40 facts, the headline recipe recalls 79–80/80 TF and GEN across three seeds with different fact values, using 245k–256k slots (0.32–0.86 MB including positions), with damage within noise of zero and bit-exact revoke. The nearest-size LoRA, rank 1 int4 (0.69 MB), recalls 75/80 on both metrics. Gaps of 1–2 items are within noise; 3–4 may matter, but all LoRA baselines remain single seed; their run-to-run spread is unmeasured. No damage difference is detectable between the best penalized FSA runs and fp32 penalized LoRA. Synthetic in-pair packed patches have no detectable serving cost; learned patches still need benchmarking. At 300 conflicting facts the FSA

recipe fails to fit its training facts and loses to LoRA. EP-1 v1 failed concept gain for every arm. These results do not establish lifelong learning or general personalization.

Contributions. (1) Quantization zeros as a removable adaptation reserve, with accounting for positions, values, scales, and inference format. (2) Real-model results comparing ternary Bop, learn-then-prune, fp4, and LoRA on recall, damage, and storage. (3) A measured KL floor under random perturbations, with routing near-ties as a possible explanation rather than an established cause. (4) A patch-revocation contract and a pre-registered multi-session concept experiment (§8).

2 Background and related work

Mask-based continual learning. PackNet [Mallya and Lazebnik, 2018] freezes kept weights and trains freed ones, recovering a task by its mask; Piggyback [Mallya et al., 2018] *learns masks over fixed weights* (it does not refill zeros); HAT [Serra et al., 2018] uses hard task-attention and reports *reduced*, not zero, forgetting; Supermasks [Zhou et al., 2019] make the mask itself the trainable object; Progressive Networks [Rusu et al., 2016] add and could drop columns. FSA is PackNet with the free support supplied by the quantization format. **Quantized continual learning.** Ada-QPackNet [Pietroń et al., 2023] and TinySubNets [Pietroń et al., 2025] already combine PackNet-style capacity reuse with quantization; SPDF [Thangarasa et al., 2023] reopens zeroed pretrained weights for dense fine-tuning. These sharply narrow our novelty; our distinct element is a reserve *pre-addressed by the deployed format* and composed by union. **Parameter-efficient and continual adapters.** LoRA [Hu et al., 2022], QLoRA [Dettmers et al., 2023], QA-LoRA [Xu et al., 2024]; Biderman et al. [2024] find LoRA learns and forgets less than full fine-tuning (a possible rank explanation, not a theorem); O-LoRA [Wang et al., 2023] does continual learning in orthogonal LoRA subspaces. **Sparse / composable adaptation.** Diff pruning [Guo et al., 2021] and composable sparse fine-tuning [Ansell et al., 2022]; union of *disjoint* sparse supports is the easy case of the latter. **Editing, memory, unlearning.** MEMIT [Meng et al., 2023] edits facts in place and can override; FSA adds paths that can suppress or correct outputs while enabled but cannot edit a base weight; GRACE [Hartvigsen et al., 2023] and WISE [Wang et al., 2024] keep deletable, routed side-memory for sequential edits while freezing the base; SISA [Bourtoule et al., 2021] is exact unlearning by shard isolation; TOFU [Maini et al., 2024] benchmarks fictitious-fact forgetting. **Personalization and retrieval.** OPPU [Tan et al., 2024] attaches a per-user PEFT module (removed by user identity, not per record); LaMP [Salemi et al., 2024] is a personalization benchmark with retrieval baselines; RAG [Lewis et al., 2020] and kNN-LM [Khandelwal et al., 2020] store facts externally and delete them by dropping a record—the baseline any "delete a fact" claim must beat. **Low-bit models.** BitNet b1.58 [Ma et al., 2024, 2025] and LittleBit [Lee et al., 2025]. Bop [Helwegen et al., 2019] optimizes binary weights without latent real weights; we extend its gradient-memory rule to ternary birth and death (§7).

3 Method

3.1 Format and reserve

In `pair8`, weights form aligned blocks of eight (four pairs); each weight is ternary times a scale, and at most two pairs per block are non-zero. A block thus has 417 reachable states ($1 + 4{\cdot}8 + \binom{4}{2}8^2$), i.e. $\log_2 417/8 \approx 1.088$ bits/weight asymptotically (1.125 at a fixed 9 bits/block), and is $\geq 50\%$ zero *by construction* (ternary alone guarantees no such fraction). **Design reserve.** The forced-zero mask selects the two pairs pair8 did not select: 4 of every 8 weights (at least half). It is deterministic and ships with the model. A patch writes only there. Filling those pairs can produce more than two nonzero pairs, which is illegal pair8. Serving requires a sparse overlay beside the packed kernel or wider block encoding; implementation and measurements are pending.

Measured variant. All reported FSA runs use in-pair slots: zeros inside already-active pairs, a different set of base zeros from the forced-zero mask (the two sets are disjoint). Filling them preserves pair occupancy; plain ternary values also preserve the codebook. Fp4 and learned scales do not. In-pair slots and forced-zero slots lie in different pairs. Filled positions and signs must still be stored for revocation; the joint rate of 1.088 bits/weight is not the marginal price of a filled slot.

3.2 Adaptation and exact revoke

Train only the free-slot patch ΔW, with support $\mathcal{R}_d \cap \mathcal{S}_0 = \varnothing$, while the base remains live. Base weights, per-row scales, router, and norms are frozen. The in-pair variant also freezes pair occupancy. A ternary writer fills masked zero slots with $\pm\alpha$, where α is the frozen row scale. The enabled model uses $W + \Delta W$; zeroing the filled slots returns every patched tensor bit-identical to W, since the patch never overwrites base support or shared state. In the 40-fact runs, patch-off logits were bitwise equal to the base on one 512-token held-out window, and test recall returned to

0/80. The 300-fact run returned to 0/600; EP-1 revoke restored its own baseline metrics.

This restores the base, without guaranteeing its behavior while the patch is on. It does not delete individual facts within a patch. If later sessions train with an earlier patch enabled, zeroing that session's slots alone need not remove its influence on retained patches or training state (§8).

3.3 Honest accounting

FSA fills existing zero weights without changing the logical element count. Only the plain ternary in-pair variant uses the same packed tensors and kernel with no added tensors or inference compute. Synthetic in-pair patches preserve file size and memory at reported precision and revoke to a byte-identical packed file (§6.5); learned patches have not been benchmarked. The forced-zero design needs a sparse overlay or wider block encoding; its serving cost, file size and memory are pending. Zeroing its mask still restores the exact base. Storage is not free: revocation needs positions and signs in a stored diff, or the original pack. The learned-block-scale arm adds parameters and storage and is outside these packed-format claims. A full per-domain mask would cost one bit per weight; sparse position coding avoids that cost for sparse patches. Frozen row scales constrain each ternary slot to $\{-\alpha, 0, +\alpha\}$, a capacity restriction. Actual position-inclusive patch sizes are reported in §6; hypothetical trit-only accounting is insufficient.

4 Real-model setup

Lambda is an 8B-parameter ternary MoE with 128 routed experts per MoE layer, 12 active per token, model width 1152, and a recurrent sequence-mixing state between layers. Each routed expert has a 2304×384 up projection, squared ReLU, and a 384×2304 down projection, stored as `pair8` trits times frozen per-row scales. All reported FSA runs use the in-pair variant, a different set of base zeros (inside the selected pairs, disjoint from the forced-zero mask) that keeps pair occupancy legal; plain ternary patches keep pair8 legal. Layers 14–15, all 128 experts and both projections provide 66,807,179 in-pair slots. Forced-zero-mask results are pending. Below, "slots" denotes the in-pair write set. Historical late-layer ablations use the same in-pair rule at their stated placement. Base support, scales, pair occupancy, router, and all other parameters remain frozen. Plain ternary births take value $\pm\alpha$ for row scale α. The block-scale arm keeps ternary signs but multiplies them by learned magnitudes, one per row and 16 input columns, initialized at 0.5α. Its values leave $\{-\alpha, 0, +\alpha\}$; the packed-format and no-new-parameter claims do not cover this arm. The fp4 variant uses magnitudes $\{0, 0.125, 0.25, 0.375, 0.5, 0.75, 1, 1.5\}\alpha$ (the E2M1 grid scaled by 0.25α), with minimum nonzero magnitude 0.125α and cap 1.5α; it leaves the ternary codebook. Evaluation decodes experts to bf16. Plain ternary in-pair patches could use the same packed kernel; synthetic patches have been benchmarked there (§6.5), while accuracy evaluation still used decoded bf16 weights.

The task has 40 synthetic facts about a fictional person (favorite cheese, boat's name, house number). Training lasts 300 steps with answer-token-only loss. Each step samples three phrasings per fact from a pool of 12 templates (three plain sentences and nine augmented forms, including "Question:/Answer:" and "User:/Assistant:"). Two further phrasings and 16×512 general-text windows provide validation monitoring and checkpoint selection. Test recall is scored under teacher forcing: given the prompt, the model's greedy (argmax) prediction must match every answer token. The new runs also measure free-running greedy generated recall: the whole answer must match at a word boundary, then a terminator or end-of-sequence within two tokens (text after that is ignored). Older runs test teacher-forced recall only. Recall is measured on two phrasings never used in training or validation: "Q: What is X's attr? A:" and "A little-known fact about X: her attr is". The Q/A form is held out verbatim but resembles trained question/answer formats. There are 80 test items; one item is 1.25 percentage points, and the unpatched model gets 0/80. Damage is mean per-token ΔNLL relative to the base on 40 disjoint 512-token WikiText windows (about 20k tokens), with paired standard error (SE) over windows. The unpatched model's NLL is 2.890 on a separate small diagnostic pool of five 512-token windows, which also provides the reference for the chaos-floor NLL ranges. All evaluations run one sequence at a time: 16-prompt batching flipped 15/32 greedy base continuations, so 1–2 item gaps are within noise. The legacy 5×512 KL/top-1 set shares 57% of its 32-token sequences with training anchors; the 40×512 ΔNLL panel, validation and conversation text have zero overlap, enforced by the harness. Each of the three headline seeds varies both fact values and training randomness: seed 1 differs from seed 0 on 33/40 attribute values, and seed 2 on 34/40. Other runs, including LoRA, use one each. Same-seed repeats do not test robustness across seeds. Table 2 reports step 300, except repair, which adds 50 steps with births disabled.

5 A chaos floor for KL

Random, untrained ternary flips produce a KL floor while NLL on the separate five-window diagnostic pool stays within about 0.01 nats of the unpatched model. For the down-projection flips in Table 1 and Figure 1, the range is about -0.010 to $+0.003$ relative to the diagnostic-pool base NLL of 2.890; across all flip settings tested, it is about

−0.010 to +0.005. A single layer-15 zero in one expert's down projection illustrates the bf16 threshold: changes at most 0.01α round away; a full trit changes about 6% of top-1 tokens. Two different single flips disagree with each other by KL 0.027, slightly more than one flip differs from the base (0.024), and the divergence is roughly flat across sequence position. These observations suggest a different but equally good trajectory after a change exceeds bf16 resolution. Near-ties in discrete MoE routing, propagated through recurrent state, are a plausible driver, not a proven explanation.

Random flips	Layer 2 KL	Layer 15 KL	Layer 28 KL
1	0.027	0.023	0.007
1,000	0.027	0.023	0.015
1,000,000	0.028	0.026	0.023

Single change	KL	Top-1 agreement
$\varepsilon \leq 0.01$	0	1.000
$\varepsilon = 0.1$	0.0105	0.978
$\varepsilon = 1$	0.0238	0.939

Table 1: Untrained perturbations (single seed). Top: random in-pair-slot flips in down projections. Bottom: one layer-15 weight changed by $\varepsilon\alpha$; small changes are absorbed by bf16 rounding.

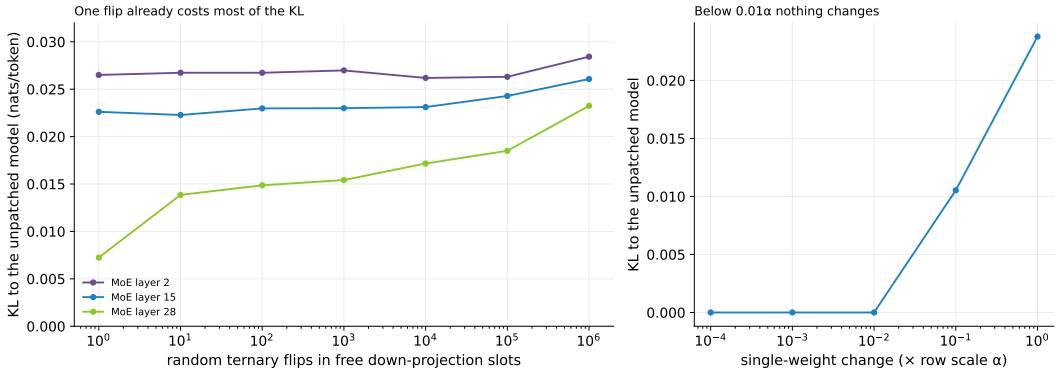


Figure 1: KL against the number of random ternary flips in in-pair down-projection slots at layers 2, 15, and 28 (left), and against a single-weight change relative to its row scale (right). Unlike layers 2 and 15, layer 28 has a lower KL of 0.007 for a single flip, rising to 0.023 at one million flips. NLL on the separate diagnostic pool of five 512-token held-out windows remains within about 0.01 nats of its base NLL of 2.890 (range about −0.010 to +0.003) for these down-projection flips; across all flip settings tested, the range is about −0.010 to +0.005.

For always-on edits the typical KL floor is about 0.02–0.027, with layer-dependent exceptions in Table 1. A KL penalty mostly fights this floor: increasing its weight from 1 to 30 left KL near 0.06 while recall fell from 48% to 20%. We therefore measure damage with paired ΔNLL, interpret KL relative to this floor, and use a local output penalty for preservation (§7).

6 Results

All reported FSA results use the in-pair variant (66,807,179 in-pair slots at layers 14–15), a different set of base zeros (inside the selected pairs, disjoint from the forced-zero mask) that keeps pair occupancy legal. Plain ternary patches keep pair8 legal; fp4 and learned scales leave its codebook. Forced-zero-mask results are pending.

6.1 Recall and damage

The earlier unbudgeted Bop + penalty run recalled 80/80 under teacher forcing, but its same-configuration rerun recalled 78/80 TF and 78/80 GEN at -0.0011 ± 0.0019 damage. The 80/80 result was not reproduced exactly:

Method	TF (of 80)	GEN (of 80)	ΔNLL ± SE
Bop + penalty + budget + rehearsal, seed 0	79	79	-0.0001 ± 0.0017
Same headline recipe, seed 1	79	79	-0.0015 ± 0.0019
Same headline recipe, seed 2	80	80	$+0.0006 \pm 0.0020$
Bop + penalty + budget + energy threshold	79	78	$+0.0013 \pm 0.0019$
Bop + penalty + budget + repair	78	78	$+0.0029 \pm 0.0016$
Bop + penalty + budget	77	76	-0.0003 ± 0.0021
Bop + penalty (rerun)	78	78	-0.0011 ± 0.0019
Bop + penalty (earlier)	80	—	$+0.0015 \pm 0.0017$
Bop $\tau = 0.7$ (run 1)	80	—	$+0.0266 \pm 0.0035$
Bop $\tau = 0.7$ (repeat)	80	—	$+0.0237 \pm 0.0036$
Bop $\tau = 0.9$	80	—	$+0.0314 \pm 0.0033$
Learn-then-prune, decay 0.3	73	—	$+0.0250 \pm 0.0032$
Learn-then-prune, decay 0.5	71	—	$+0.0129 \pm 0.0028$
Learn-then-prune, decay 0.1	75	—	$+0.0667 \pm 0.0035$
Signs + learned block scales, decay 0.3	70	—	$+0.0282 \pm 0.0031$
Fp4 + penalty	77	—	$+0.0050 \pm 0.0015$
LoRA $r = 1$ + penalty, fp32 (baseline)	75	76	-0.0002 ± 0.0019
LoRA $r = 2$ + penalty, fp32 (baseline)	77	76	$+0.0019 \pm 0.0014$
LoRA $r = 4$ + penalty, fp32 (baseline)	77	76	-0.0001 ± 0.0020
LoRA $r = 16$ + penalty (baseline)	78	—	$+0.0005 \pm 0.0018$
LoRA $r = 16$, unregularized (baseline)	78	—	$+0.9361 \pm 0.0302$

Table 2: Final endpoints on the real model: three headline seeds with different fact values, one seed for other runs. TF: teacher-forced recall; GEN: generated recall with a clean stop; —: not measured. All FSA rows use ternary slots except fp4. The budget is 300k slots; FSA penalty is $\lambda = 10$, LoRA penalty $\lambda = 100$. Repair reports 300 training plus 50 death-only steps; all other endpoints are step 300. Every 40-fact run in this table restores bitwise-equal logits on one 512-token window and test recall to 0/80.

LoRA baseline	Precision	TF (of 80)	GEN (of 80)	ΔNLL ± SE
$r = 1$ + penalty	bf16	76	75	$+0.0040 \pm 0.0020$
$r = 1$ + penalty	int8	76	76	$+0.0049 \pm 0.0018$
$r = 1$ + penalty	int4	75	75	$+0.0021 \pm 0.0017$
$r = 2$ + penalty	bf16	76	77	$+0.0018 \pm 0.0017$
$r = 2$ + penalty	int8	77	76	$+0.0013 \pm 0.0014$
$r = 2$ + penalty	int4	78	76	$+0.0021 \pm 0.0017$
$r = 4$ + penalty	bf16	78	76	-0.0010 ± 0.0018
$r = 4$ + penalty	int8	77	76	$+0.0015 \pm 0.0017$
$r = 4$ + penalty	int4	78	77	$+0.0003 \pm 0.0016$

Table 3: Measured factor-precision evaluations of the same single-seed LoRA adapters, without retraining. Int8/int4 are fake-quantized, with one bf16 absmax scale per rank vector. Quantized accuracy and damage were measured; LoRA seed spread is unmeasured.

variation between executions is a couple of items on one seed, within noise. The two plain Bop $\tau = 0.7$ executions also use the same seed and protocol; the repeat listed decay, but Bop has no real-valued weights to decay, so it is the same configuration. This variation does not establish robustness across seeds. Earlier validation-selected checkpoints remain distinct from the final endpoints reported here.

6.2 Storage

These estimates cover the in-pair variant; forced-zero storage and serving costs are pending.

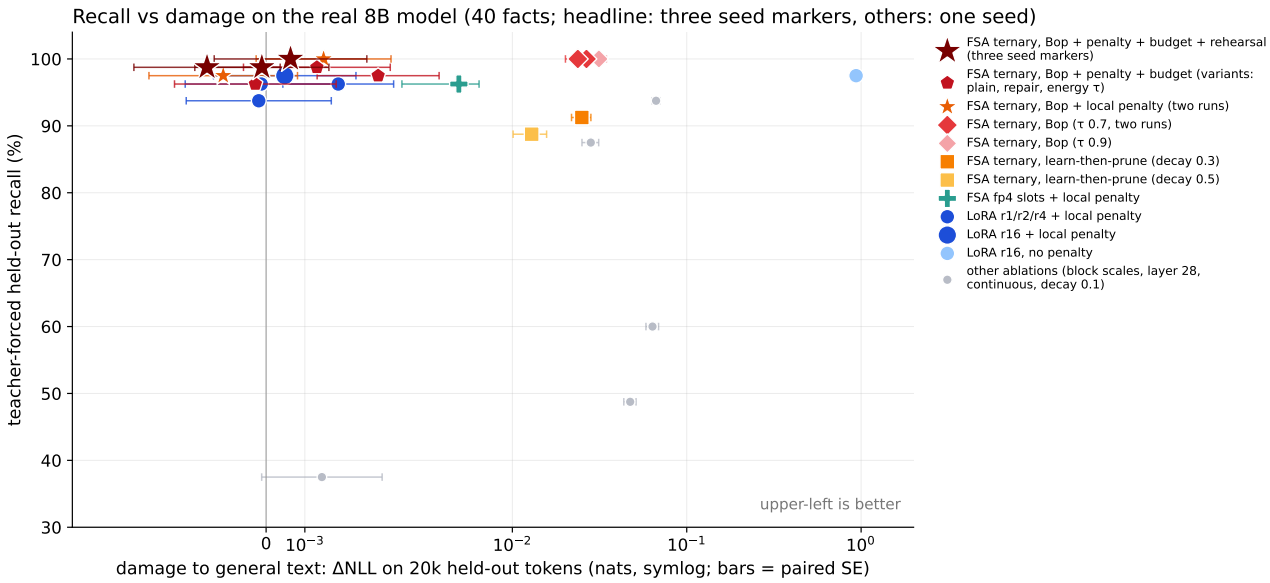


Figure 2: Teacher-forced held-out recall versus general-text ΔNLL, with paired-SE bars. The symmetric log axis is linear near zero, preserving negative estimates and their uncertainty. The headline has three individual seed markers with different fact values, each with its paired SE; other runs have one seed. All use 12 training formats, two held-out phrasings and 40 disjoint 512-token WikiText windows. Minor ablations include block scales and layer-28 down-only patches.

Patch	Slots / parameters	E MB	I MB
Bop + penalty + budget + rehearsal, seed 0	249,755	0.33	0.84
Same headline recipe, seed 1	245,281	0.32	0.83
Same headline recipe, seed 2	255,585	0.33	0.86
Bop + penalty + budget + energy threshold	238,662	0.32	0.81
Bop + penalty + budget + repair	250,538	0.33	0.85
Bop + penalty + budget	243,935	0.32	0.82
Bop + penalty (rerun)	735,955	0.82	2.48
Bop + penalty (earlier)	739,549	0.83	2.50
Bop $\tau = 0.7$ (run 1)	812,420	0.89	2.74
Bop $\tau = 0.7$ (repeat)	669,705	0.76	2.26
Bop $\tau = 0.9$	588,673	0.68	1.99
Learn-then-prune, decay 0.3	542,205	0.64	1.83
Learn-then-prune, decay 0.5	523,229	0.62	1.77
Learn-then-prune, decay 0.1	1,857,504	1.76	6.27
Signs + learned block scales, decay 0.3	1,125,769	1.17 + scales	3.80 + scales
Fp4 + penalty	21,265,606	18.17	79.75
LoRA $r = 1$ (baseline)	1,376,256	2.75 / 1.38 / 0.69	
LoRA $r = 2$ (baseline)	2,752,512	5.51 / 2.76 / 1.38	
LoRA $r = 4$ (baseline)	5,505,024	11.01 / 5.51 / 2.76	
LoRA $r = 16$ (baseline)	22,020,096	44.04 bf16	

Table 4: Position-inclusive estimates, decimal MB (three headline seeds with different fact values, other runs single seed). E: entropy-coded positions plus signs over 66,807,179 in-pair slots; I: 26-bit index plus value (1 bit ternary, 4 bits fp4). LoRA ranks 1–4 list bf16 / int8 / int4, including bf16 scales for int8/int4, with measured accuracy at each precision. Rank 16 lists bf16 size with accuracy of fp32 factors; no bf16 evaluation is available. Block scales add about 113 MB. Fp32 training factors use twice the bf16 bytes.

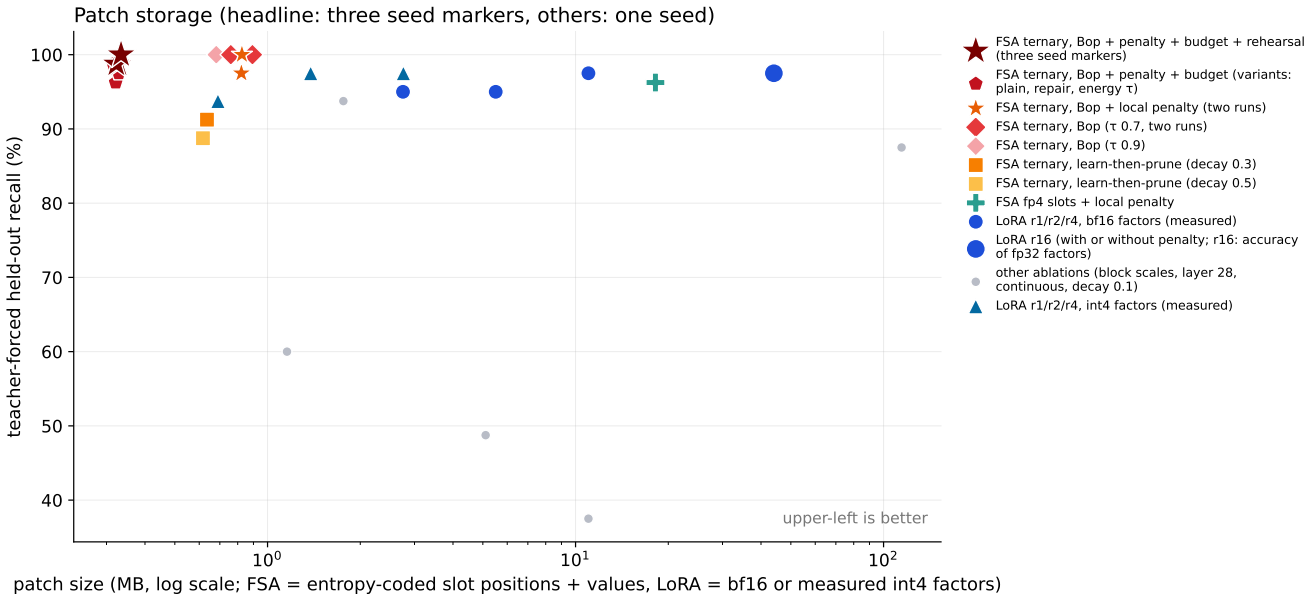


Figure 3: Teacher-forced recall versus position-inclusive storage. Three individual markers show the headline seeds, with different fact values; other runs have one seed each. FSA uses entropy-coded positions and values, over each run's own reserve. LoRA ranks 1–4 show measured bf16 recall (95%, 95%, 97.5%) at bf16 size and measured int4 points; r16: accuracy of fp32 factors, shown at bf16 size because no bf16 evaluation is available. Minor layer-28 ablations have a 16.8M-slot in-pair reserve; learned block scales add about 113 MB in fp32, bringing that ablation to about 114 MB.

6.3 Honest reading

Across three seeds with different fact values, the headline's mean TF and GEN are 99.2% (range 98.75–100%), mean ΔNLL is −0.0003 (range −0.0015 to +0.0006), and patches use 245k–256k slots. FSA recalls 79–80/80 on all three seeds with different fact values versus 75–78/80 TF and 75–77/80 GEN across LoRA ranks 1–4 and measured precisions, at 0.69–11.01 MB (int4 to bf16). The nearest LoRA size is rank 1 int4, 0.69 MB: 75/80 on both metrics. FSA's two encodings bracket that size rather than match its bytes exactly. Gaps of 1–2 items are within noise; 3–4 may matter, but LoRA ranks 1/2/4/16 remain single seed; their run-to-run spread is unmeasured. FSA does not beat LoRA overall on this evidence, and loses at 300 facts below.

The best penalized FSA runs, with or without rehearsal, and fp32 penalized LoRA all have $|\Delta\text{NLL}| \leq 0.003$ with SE around 0.002: at this sample size no damage difference is detectable between them. This summary does not cover rank-1 bf16/int8 evaluations, whose measured damage is +0.0040/ + 0.0049 (Table 3). Plain Bop without the penalty costs +0.024 to +0.031; those rows remain as history. Rehearsal uses the same corpus as the damage panel, with no shared 32-token sequences, so near-zero damage is measured in-distribution; other kinds of text are untested. Removing the patch restores the base exactly, without establishing preservation while enabled. EP-1 v1 failed concept gain in every arm (§8).

On the 40-fact in-conversation test, the question follows about 545 tokens of unrelated chat. The headline achieves 85% generated recall versus 40% for the unbudgeted base (single seed, 40 items). Short-prompt recall does not distinguish them beyond noise.

6.4 Many conflicting facts: a capacity limit

The FSA run uses the same layers 14–15 in-pair reserve; forced-zero results are pending. Three fictional people each have 100 facts, with 30 attributes shared by all three and conflicting values across people for every fact. Training samples 40 facts per step for 600 steps; testing uses 600 held-out items. Both runs use one seed. The FSA recipe is the headline recipe with its budget raised to 1M slots; LoRA uses rank 16 and penalty 100. FSA fits only 22.9% of training facts, gives another person's value on conflicting attributes 14.2% of the time, and reaches 8.3% teacher-forced in-conversation recall. It fails to fit even its training facts: this is a capacity or optimization limit, not only a generalization gap. Its support never settled: 18.8M births, 12.2M deaths and 5.7M evictions produced an 875k

Method	TF	GEN	ΔNLL ± SE
FSA, 875,105 filled slots	18.2%	18.2%	$+0.0046 \pm 0.0024$
LoRA, fp32 factors	63.0%	65.7%	$+0.0017 \pm 0.0023$
LoRA, measured bf16	64.3%	64.3%	$+0.0034 \pm 0.0018$
LoRA, measured int8	64.7%	65.2%	$+0.0027 \pm 0.0018$
LoRA, measured int4	64.8%	66.3%	$+0.0029 \pm 0.0020$

Table 5: Final 300-fact endpoints. LoRA has 22,020,096 parameters. Its final evaluation is complete; values are from the run log because the result file awaits checkpoint-selection evaluation. Earlier checkpoint evaluations are excluded. Quantized factor recall and damage are measured.

patch. LoRA does much better, and its factors quantize to int4 without loss on this task. FSA reruns with 2M- and 4M-slot budgets at 1,200 steps are running; results are pending.

6.5 Serving

All serving measurements here cover synthetic ternary in-pair patches, including same-kernel, file-size, memory and byte-identical packed-file revoke checks. The forced-zero design needs a sparse overlay or wider block encoding; that path is not built or benchmarked, and serving cost, file size and memory are pending. Synthetic in-pair patches use uniformly drawn slots with random signs at the size of the earlier Bop patches (670k) and a 3M stress test. They were written into the released packed model and benchmarked on Mac Metal, Mac CPU and a Snapdragon 8 Elite phone; learned patches (now about 250k slots) have not been benchmarked. The table gives paired patched/base throughput medians. Paired median ratios stay within about 2% of 1.0, with no consistent direction; ranges are min–max over rounds. For Metal text-prefill, the ranges touch 1.0 ([0.993, 1.000] and [0.991, 1.000]). The Mac model

Device / backend	Decode, 670k	Decode, 3M	Prefill, 670k	Prefill, 3M
Mac M5 Max, Metal	0.999	0.999	0.999	1.001
Mac M5 Max, CPU, 8 threads (table lookup)	1.005	0.990	1.002	1.009
Mac M5 Max, CPU, 8 threads (alternate lookup)	1.006	0.998	1.004	0.999
Snapdragon 8 Elite, CPU, Q8 trunk	1.002	0.982	0.996	1.000

Table 6: Synthetic in-pair packed-patch serving ratios (forced-zero serving pending), paired within rounds. Median ratios stay within about 2% of 1.0, with no consistent direction. No slowdown was detected; min–max ranges over rounds are not a significance test and do not prove zero cost. Learned patches are not yet benchmarked.

file is 2,133,006,592 B and the phone's Q8-trunk file is 1,656,608,512 B; patching leaves each file's size unchanged. Peak memory is unchanged to reported precision, about 2.75 GiB on Metal, 3.29 GiB on Mac CPU and 3.24 GB on the phone. Phone readings span 3.244–3.245 GB. In-pair revocation restores byte-identical packed files, checked by SHA-256. Accuracy evaluation still used decoded bf16 weights. Synthetic patches test kernel cost, not learned-patch accuracy or routing effects. LoRA is an analytic estimate only in this runtime: rank 16 would add about 0.5–1.5% decode cost and 44 MB per adapter; merging would densify 64 MB of packed experts to 906 MB bf16. Switching a running model's patch is not yet implemented; today switching requires changing the packed model and reloading. One model copy serves one user's patch at a time, unlike multi-adapter LoRA batching.

7 Writers

7.1 Ternary Bop

Bop is Helwegen et al.'s binary optimizer [Helwegen et al., 2019]. Our ternary extension adds the birth and death rules below and resets the gradient EMA after every change. Each in-pair slot holds $q_j \in \{-1, 0, +1\}$ directly, with no latent real weight. For loss gradient g_j, maintain

$$m_j \leftarrow (1-\gamma)m_j + \gamma g_j, \qquad \gamma = 0.05.$$

If $q_j = 0$ and $|m_j| > \tau$, birth sets $q_j \leftarrow -\operatorname{sign}(m_j)$. If $q_j \neq 0$, $|m_j| > \tau$, and $\operatorname{sign}(m_j) = q_j$, death sets $q_j \leftarrow 0$: the gradient says the current sign hurts. Reset m_j to zero after each change. Thresholds are 0.7 or 0.9 times the RMS

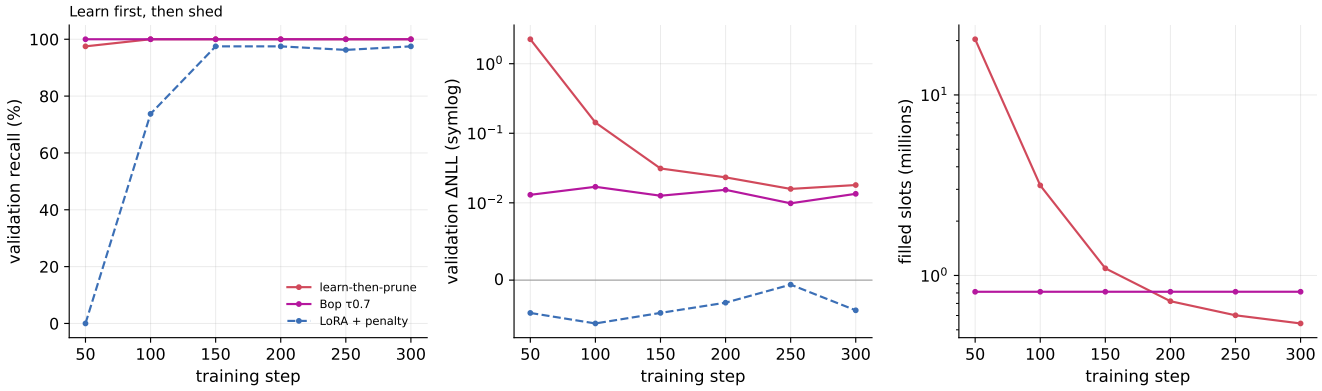


Figure 4: Validation recall, validation ΔNLL (symmetric log scale), and filled slots (millions, log scale) across training. The recall and damage panels show learn-then-prune, ternary Bop $\tau = 0.7$ run 1, and regularized LoRA; learn-then-prune uses decay 0.3. The filled-slot panel shows only those two FSA writers; the new repeat and decay 0.5 trajectories are not plotted. Bop's filled-slot count is set by its threshold early and then stays flat, while learn-then-prune fills many slots first and sheds them over training. Validation trajectories differ from the held-out test endpoints in Table 2. These historical trajectories each use one seed; they do not plot the headline seeds.

of the first answer-loss gradient over in-pair slots. Historical plain Bop runs use no penalty; new runs add the local penalty below. Bop does not use weight decay. The threshold controls flips: small gradients need not become full $\pm\alpha$ steps, unlike Adam's per-coordinate normalization, which can turn persistent small gradients into full flips. In the historical single-seed runs, slots are filled early: 812,077 slots were filled at step 50 (812,420 at step 300) at $\tau = 0.7$. Higher τ uses fewer slots (589k) at slightly higher damage, so it is not a simple damage dial. The $\tau = 0.7$ repeat has 669,465 filled slots at step 50 and 669,705 at step 300; its revoke is also bit-exact, with test recall returning to 0/80.

7.2 Learn-then-prune

These historical comparisons use a single seed per run. Each in-pair slot has an fp32 master w_j in row-scale units. Forward uses $q_j = \text{sign}(w_j)\text{I}(|w_j| > 0.5)$, where I is the indicator; gradients pass straight through [Bengio et al., 2013]. AdamW [Loshchilov and Hutter, 2019], learning rate 0.02, applies decoupled weight decay 0.3 to masters. The model learns early: at step 50 validation recall is 97.5%, with 20.3M filled slots and validation ΔNLL +2.2. Decay pulls weakly supported masters under threshold: 3.2M slots at step 100, 1.1M at 150, and 0.54M at 300. Validation damage falls to about +0.02 and validation recall stays 100%; test recall is separately 91.25%. Held-out damage fell from +0.0282 at step 250 to +0.0250 at step 300, while validation damage rose slightly (+0.016 → +0.018). With decay 0.5, filled slots fall from 14.18M at step 50 (98.75% validation recall) to 1.12M at step 100, 0.62M at 150, 0.50M at 200, and 0.45M at 250 (92.5% validation recall), then rise to 0.52M at 300 (97.5% validation recall). Test recall at step 300 is 88.75%; revoke is bit-exact. Whether longer pruning helps is untested.

7.3 Fp4 and the local output penalty

Four-bit float slots times α use AdamW decay 0.1 and add the general-text penalty

$$\lambda \frac{\sum_{l,t} \|x_t \Delta W_l^T\|^2}{\sum_{l,t} \|x_t W_l^T\|^2 + 10^{-12}}, \qquad \lambda = 10,$$

where l ranges over patched linear projections, t over general-text tokens, x_t is the input to that projection, W_l is its frozen base weight, and ΔW_l is the patch. The denominator, base-weight output energy on the current inputs, is treated as a constant normalizer and is not differentiated. This smooth local penalty measures each patched projection's direct output change rather than the change in final logits, so it is not dominated by the whole-model divergence behind the KL floor. Its inputs still come from the patched forward pass: earlier patched layers can change later inputs and routing. The best LoRA run uses the same penalty with $\lambda = 100$. At that strength the penalty stopped earlier ternary learning: its gradient is zero at zero, yet one birth costs a full $\alpha^2\mathbb{E}[x^2]$. Bop with $\lambda = 10$ now learns while keeping measured damage within noise of zero (Table 2).

7.4 Budget and preservation variants

Gain-minus-cost budget. At most 300k slots may be filled. The allowed set is re-chosen every 10 steps up to step 200 (the harness default; that setting is not stored in the result files), by each slot's gradient evidence minus its expected general-text cost, α^2 times the input column's general-text energy. Births go where evidence for the facts is strongest relative to expected general-text change. **Rehearsal.** The headline objective is 0.8 times fact loss plus 0.2 times ordinary next-token loss on general WikiText windows, alongside the local penalty. Those windows share no 32-token sequences with the damage panel but come from the same corpus. Near-zero damage is therefore measured in-distribution; other kinds of text remain untested. **Energy-scaled threshold.** The row's birth threshold is multiplied by $\mathrm{clamp}((e_{\mathrm{row}}/\operatorname{median} e)^{0.5}, 0.25, 4)$, where e_{row} is the expected general-text output energy of one birth in that row and the median is over rows with in-pair slots and positive energy. Sensitive rows need more evidence. **Repair.** After 300 training steps, 50 more Bop steps keep the local penalty at $\lambda = 10$ and disable births, allowing deaths only. Rehearsal has three seeds with different fact values; other variants have one each. Recall differences of 1–2 items are within noise and do not establish their relative value.

7.5 Ingredients that mattered

Answer-only loss avoids learning template boilerplate, which caused most early damage with whole-sentence loss. Twelve diverse formats, including Q/A, largely closed the validation-to-test gap (Bop and penalized LoRA test at their validation recall; learn-then-prune decay 0.3 remains 100% validation vs 91.25% test). An earlier protocol reached only 54%/56% held-out recall. The broad footprint mattered: 32 selected experts in one layer, down only, with a 500k-slot budget (350k–404k slots filled) starved capacity (19–29% recall). Damage uses the roughly 20k-token held-out pool, with checkpoint selection on a separate validation pool.

7.6 What did not work

KL penalties fight the chaos floor and cost recall. Ternary slots with smooth penalty and decay peak in recall, then are pruned away. Tiny fp4 slots in late layer 28, down only, with a 2M-slot budget and grid magnitudes starting at 0.03125α, fill about 2.0M slots but yield 0% recall at +0.0067 damage: the patch changed the model but learned none of the tested answers. Patch-only learned block scales are no better than plain ternary (87.5%/+0.028) and add substantial storage. Single-late-layer continuous bf16 slots give 48.75%/+0.047, versus same-placement LoRA's 37.5%/+0.0014: a recall–damage trade, not an overall improvement over LoRA. Gain-minus-cost ternary slot selection at that placement gives 60%/+0.064.

Churn controls also failed to improve on the headline. Against the unbudgeted, unrehearsed Bop $\tau = 0.7$ + penalty 10 base (78/80 TF and GEN, -0.0011 ± 0.0019, 735,955 slots), preventing death for 20 steps after birth gives 80/80 on both metrics, $+0.0024 \pm 0.0017$, and 543,145 slots. A 50-step grace gives 79/80, $+0.0052 \pm 0.0017$, and 769,165 slots; requiring 1.5 times the birth threshold for death gives 79/80, $+0.0029 \pm 0.0019$, and 783,165 slots. Recall differs by only 1–2 items, within noise, while damage is slightly higher than the base; grace 50 is about three SE above zero. None is an improvement.

Post-training utility pruning ranked slots by a first-order estimate and tried 10%, then 5%, then 2.5% chunks, accepting only if validation recall held. The headline accepted no chunk: 0% removed from 249,755 slots. Removing the "least useful" 10% raised fact loss despite a predicted decrease, and validation TF recall fell to 81%. The unbudgeted base accepted one 2.5% chunk (717,557 slots kept), but test TF fell from 78/80 to 75/80 and GEN from 78/80 to 77/80 (the one-item GEN drop is within noise). The first-order estimate is wrong at a trained minimum (fact loss ≈ 0.01): gradients are near zero, while reverting a flip is a full-α step. Reverting the predicted least-useful 10% raised base fact loss by +0.258 versus a predicted -0.025; for the headline, +0.150 versus -0.013. Just 7.5% of the base patch's flips carry half the positive first-order utility. The failure diagnoses the estimate, rather than proving there is no removable slack. Exact leave-out by expert and by layer is running.

8 Toward epigenesis: multi-session learning

The in-pair ternary reserve now carries facts on a real model (§4). A revocable domain patch is therefore demonstrated for this task: enable the slice for its domain; disable it to recover the base. The personal-model application remains an untested proposal, with a narrower contract:

> **Patch-revocation contract.** Deleting an isolated patch exactly removes *that patch's parameter contribution*. Equivalence to a model trained without its data additionally requires that no retained patch or training state depended on it. It does not delete individual facts (facts within a slice are superimposed), and it does not make information already in the base inaccessible.

Retrieval deletes individual records while leaving the base unchanged [Lewis et al., 2020, Khandelwal et al., 2020]; disabling a per-user adapter removes that user's contribution [Tan et al., 2024]. The case for weights is skills, style, and policy that benefit from adaptation in every forward pass. Arbitrary facts remain a retrieval baseline's natural task. The real ternary fact experiment offers smaller patches, with no detectable damage difference between the best penalized FSA runs and fp32 penalized LoRA at this sample size. General personalization remains untested.

EP-1 v1 uses the in-pair write set; its fp4 and learned-scale arms leave the original codebook. It completed five sessions, distilling each session's self-study into a patch. The synthetic user contains 40 policies, 100 recurrent facts (each in three sessions), and 100 one-off lure facts (20 per session). On 100 four-way test concept items, the unpatched model scores 0.24 (chance 0.25). The thesis required at least +10 concept accuracy points over the base

Arm	Session-5 concept accuracy	Gain over base
Ternary in-pair slots with learned block scales	0.32	+8 points
Fp4 slots	0.32	+8 points
Ternary Bop (added after pre-registration)	0.32	+8 points
LoRA	0.28	+4 points

Table 7: EP-1 v1, single-seed pilot. Bop was added after pre-registration with owner approval. Every arm failed only the concept-gain criterion; the multi-session thesis is not established.

after session 5, with lure, damage, retention and revoke criteria. Every arm, including LoRA, failed concept gain; all other criteria passed. Lure recall is 0/50 in every arm, with no upward ratchet. Damage bounds and retention passed; revoke restored bitwise weights, legal format and identical logits, and cleared operational state. Session-5 damage is within noise of zero for the ternary arms; fp4 is $+0.0074 \pm 0.0020$. The pre-registered ternary arm learns block scales and is outside the original packed-format and no-new-parameter claims; fp4 also leaves the ternary codebook.

The in-context ceiling measured in v1 (1.00) was invalid: each probe's context held only the correct policy's source episode, so it contained the answer. With all four rules of the category in context, the unpatched model picks the right action only 0.30 of the time (ticket 0.27, Q/A 0.34; chance 0.25). The identifiability gate was therefore invalid and the self-distillation teacher was never competent on the probed skill, so v1 cannot test whether slots consolidate concepts. EP-1 v2.1 uses coarser categorical rules and a full-table context, or a ground-truth supervised target if the teacher still fails; the multi-session thesis remains open. The 300-fact failure also calls for budget-scaled reruns and better optimization. LoRA seed spread, learned-patch serving and broader preservation need measurement. Per the pre-registration, one seed is a pilot; three are needed to claim. On-device training is not established: optimizer state can be large even when a final patch is small. A broader evaluation should include local retrieval and per-user adapters on personalization benchmarks [Salemi et al., 2024], with unlearning measured [Bourtoule et al., 2021, Maini et al., 2024].

9 Limitations

Damage and recall. The best penalized FSA runs and fp32 penalized LoRA are within noise of zero on 40 WikiText windows, without a detectable damage difference at this sample size. Plain Bop costs +0.024 to +0.031. Damage on other corpora and broader behavior is untested; rehearsal windows are sequence-disjoint but drawn from the same corpus as the damage panel. Generated recall is now measured for the new runs, checking unaided answer production and a clean stop; older runs remain teacher-forced only. Eval batch composition changes this model's outputs (16-prompt batching flipped 15/32 greedy base continuations); all evaluations run one sequence at a time, and 1–2 item recall gaps are within noise. The legacy 5×512 KL/top-1 set overlaps training anchors (57% of 32-token sequences), so it is not held out for penalty runs. The 40×512 ΔNLL panel, validation and conversation text have zero overlap, enforced by the harness.

Evaluation scope and capacity. The 40-fact headline has three seeds with different fact values, 79–80/80 TF and GEN, with damage within noise of zero; other runs, including LoRA, have one seed each. Gaps of 1–2 items are within noise; 3–4 may matter, but LoRA's run-to-run spread is unmeasured. The Q/A test form resembles training; writer and regularization choices differ. At 300 conflicting facts, FSA reaches only 18.2% recall and fits 22.9% of training facts, versus LoRA's 63.0% TF fp32 / 64.8% int4. This capacity or optimization failure limits the current recipe; FSA reruns with 2M- and 4M-slot budgets at 1,200 steps are running, with results pending. EP-1 v1 failed concept gain for every arm, including LoRA; v2 is being designed. The chaos-floor explanation is a hypothesis, and the roughly 20k-token damage pool cannot establish broad behavioral safety.

Revoke scope. Removing the whole patch restores the base bit-for-bit. This does not guarantee preservation

while the patch is on, deletion of individual facts, removal of information already in the base, or removal of an earlier session's influence after later sessions trained on top of it. The frozen base can be augmented or suppressed through new paths but its existing weights cannot be refined.

Format and deployment. Plain ternary in-pair writers stay legal pair8, but evaluation decoded experts to bf16; packed-kernel serving is measured for synthetic legal patches only, with learned patches not yet benchmarked. Fp4 and learned-block-scale variants leave the original codebook. Forced-zero serving needs a sparse overlay or wider block encoding; serving cost, file size and memory are pending. Optimizer state can be large even when the final patch is small, and on-device training is not established.

Privacy. Knowledge in weights can be elicited. A per-user model is the proposed deployment for the personal application; its safety is untested, and the patch mechanism supplies no privacy guarantee.

10 Conclusion

FSA targets the deterministic pair8 forced-zero mask; zeroing it restores the exact base. Its sparse-overlay or wider-block serving path is not built or benchmarked; serving cost, file size and memory are pending. All reported FSA results use the in-pair variant. Only plain ternary in-pair patches retain the same packed kernel; unchanged file size and memory and byte-identical packed-file revoke were measured on synthetic in-pair patches. On a real 8B ternary MoE, Bop with a local penalty, a slot budget and rehearsal recalls 79–80/80 TF and GEN across three seeds with different fact values: mean 99.2% (range 98.75–100%), mean ΔNLL -0.0003 (range -0.0015 to $+0.0006$), and 245k–256k slots (0.32–0.86 MB including positions). All three revoke bit-exactly. LoRA baselines remain single seed, leaving their run-to-run spread unmeasured. No damage difference is detectable between the best penalized FSA runs and fp32 penalized LoRA at this sample size, on the same corpus used for rehearsal. Synthetic in-pair packed patches have no detectable serving cost; learned-patch serving and broader preservation remain untested.

The recipe fails at 300 conflicting facts: 18.2% recall and only 22.9% training-fact recall, versus LoRA's 63.0% TF fp32 / 64.8% int4. EP-1 v1 failed the pre-registered concept-gain criterion for every arm, including LoRA. Multi-session consolidation remains open; v2 is being designed, alongside running 2M- and 4M-slot-budget fact reruns at 1,200 steps (results pending) and LoRA seeds. These results establish small, revocable fact patches under the measured protocol, not the lifelong-learning thesis.

References

Alan Ansell, Edoardo Maria Ponti, Anna Korhonen, and Ivan Vulić. Composable sparse fine-tuning for cross-lingual transfer. In *ACL*, 2022.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv:1308.3432*, 2013.

Dan Biderman, Jacob Portes, et al. LoRA learns less and forgets less. *TMLR*, 2024. arXiv:2405.09673.

Lucas Bourtoule et al. Machine unlearning. *IEEE S&P*, 2021. arXiv:1912.03817.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *NeurIPS*, 2023. arXiv:2305.14314.

Demi Guo, Alexander M. Rush, and Yoon Kim. Parameter-efficient transfer learning with diff pruning. *ACL*, 2021. arXiv:2012.07463.

Thomas Hartvigsen et al. Aging with GRACE: Lifelong model editing with discrete key-value adaptors. *NeurIPS*, 2023. arXiv:2211.11031.

Koen Helwegen, James Widdicombe, Lukas Geiger, Zechun Liu, Kwang-Ting Cheng, and Roeland Nusselder. Latent weights do not exist: Rethinking binarized neural network optimization. In *NeurIPS*, 2019. arXiv:1906.02107.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022. arXiv:2106.09685.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. *ICLR*, 2020. arXiv:1911.00172.

Banseok Lee et al. LittleBit: Ultra low-bit quantization via latent factorization. *NeurIPS*, 2025. arXiv:2506.13771.

Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020. arXiv:2005.11401.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019. arXiv:1711.05101.

Shuming Ma, Hongyu Wang, et al. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv:2402.17764*, 2024.

Shuming Ma et al. BitNet b1.58 2b4t technical report. *arXiv:2504.12285*, 2025.

Pratyush Maini et al. TOFU: A task of fictitious unlearning for LLMs. *COLM*, 2024. arXiv:2401.06121.

Arun Mallya and Svetlana Lazebnik. PackNet: Adding multiple tasks to a single network by iterative pruning. In *CVPR*, 2018. arXiv:1711.05769.

Arun Mallya, Dillon Davis, and Svetlana Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In *ECCV*, 2018. arXiv:1801.06519.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. *ICLR*, 2023. arXiv:2210.07229.

Marcin Pietroń, Kamil Faber, Dominik Żurek, and Roberto Corizzo. TinySubNets: An efficient and low-capacity continual learning strategy. *AAAI*, 2025. arXiv:2412.10869.

Marcin Pietroń et al. Ada-QPacknet – adaptive pruning with bit width reduction as an efficient continual learning method without forgetting. *ECAI*, 2023. arXiv:2308.07939.

Andrei A. Rusu et al. Progressive neural networks. *arXiv:1606.04671*, 2016.

Alireza Salemi et al. LaMP: When large language models meet personalization. *ACL*, 2024. arXiv:2304.11406.

Joan Serra, Didac Suris, Marius Miron, and Alexandros Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task. In *ICML*, 2018. arXiv:1801.01423.

Zhaoxuan Tan et al. Democratizing large language models via personalized parameter-efficient fine-tuning. *EMNLP*, 2024. arXiv:2402.04401.

Vithursan Thangarasa et al. SPDF: Sparse pre-training and dense fine-tuning for large language models. *UAI*, 2023.

Peng Wang et al. WISE: Rethinking the knowledge memory for lifelong model editing of large language models. *NeurIPS*, 2024. arXiv:2405.14768.

Xiao Wang et al. Orthogonal subspace learning for language model continual learning. *EMNLP Findings*, 2023. arXiv:2310.14152.

Yuhui Xu et al. QA-LoRA: Quantization-aware low-rank adaptation of large language models. *ICLR*, 2024. arXiv:2309.14717.

Hattie Zhou, Janice Lan, Rosanne Liu, and Jason Yosinski. Deconstructing lottery tickets: Zeros, signs, and the supermask. *NeurIPS*, 2019. arXiv:1905.01067.